

PLC Assignment 2 – Complete Exam Cheatsheet

Statically-Typed Interpreted Language – Python Compiler

• May 10, 2026

Architecture

Pipeline: chars → tokens → AST → types → run

1. `lexica.py` – Lexer
2. `parser.py` – Parser
3. `type_checker.py` – TypeChecker
4. `interpreter.py` – Interpreter

Support files: `tokens.py` (Token-Type enum), `ast_nodes.py` (data-classes), `symbol_table.py` (scoped symbols), `ast_printer.py` (visitor printer – must update for new nodes!), `parse_tree_printer.py` (concrete tree), `pipeline.py` (orchestration).

```
# components/pipeline.py --
run_pipeline()
tokens = Lexer(source).tokenize()
tree = Parser(tokens).parse()
scope = TypeChecker().check(tree)
outs = []
Interpreter(outs.append).execute(tree)
```

Lexer – `lexica.py`

Methods

```
class Lexer:
    def tokenize() -> list[Token] # entry
    def _scan_token() -> Token|None
    def _string() -> Token
    def _number() -> Token
    def _identifier() -> Token
    def _advance() # consume, return char
    def _match(c) # consume if peek==c
    def _peek() # look-ahead 1, no consume
    def _peek_next() # look-ahead 2
    def _make_token(tt) -> Token
    # State: self.start, .current, .line, .column, .token_column
```

KEYWORDS dict – in `lexica.py`!

```
# NOT in tokens.py -- lives in lexica.py
KEYWORDS: dict[str, TokenType] = {
    "if": TokenType.IF,
    "else": TokenType.ELSE,
    "while": TokenType.WHILE,
    "def": TokenType.DEF,
    "return": TokenType.RETURN,
    "true": TokenType.BOOL_LITERAL,
    "false": TokenType.BOOL_LITERAL,
}
```

Float-op disambiguation

```
# lexica.py uses _ARITH_DOT dict:
_ARITH_DOT = {
    '+': (PLUS, PLUS_DOT),
    '-': (MINUS, MINUS_DOT),
    '*': (STAR, STAR_DOT),
    '/': (SLASH, SLASH_DOT),
}
# In _scan_token for +/-/*/:
int_tok, float_tok = _ARITH_DOT[char]
if peek()=='.' and not peek_next().isdigit():
    advance() # consume '.'
    return make_token(float_tok) # e.g. PLUS_DOT
return make_token(int_tok) # e.g. PLUS
```

Token types

Group	Types
Literals	INT_LITERAL FLOAT_LITERAL BOOL_LITERAL STRING_LITERAL
Int arith	PLUS MINUS STAR SLASH
Float arith	PLUS_DOT MINUS_DOT STAR_DOT SLASH_DOT
Compare	EQUAL_EQUAL BANG_EQUAL
Assign	ASSIGN
Delims	LPAREN RPAREN LBRACE RBRACE COMMA SEMICOLON
Keywords	IF ELSE WHILE DEF RETURN
Identifier	IDENTIFIER
End	EOF

Error format: [line X, col Y]
LexerError: msg

Parser – `parser.py`

Grammar (BNF)

```
program -> statement* EOF
statement ->
    IDENT "=" expr ";" # assignment
    "if" "(" expr ")" block ["else" block]
    "while" "(" expr ")" block
    "def" IDENT "(" params ")" block
    "return" expr ";"
    expr ";" # fn call
    stmt
block -> "{" statement* "}"
params -> IDENT ("," IDENT)*
args -> expr ("," expr)*

# Precedence: low --> high
expr -> additive ("==" | "!=") additive?
additive -> term (("+" | "-" | "+" | "-") term)*
term -> factor (("*" | "/" | "*" | "/" ) factor)*
factor -> "-" factor | "-" factor
| INT | FLOAT | BOOL | STRING
| IDENT "(" args ")"
| IDENT
| "(" expr ")"
```

Actual `_statement()` dispatch

```
def _statement(self) -> ASTNode:
    tok = self._peek()
    if tok.type == DEF: return _func_def()
    if tok.type == IF: return _if_stmt()
    if tok.type == WHILE: return _while_stmt()
    if tok.type == RETURN: return _return_stmt()
    # assignment: IDENT then ASSIGN
    if (tok.type == IDENTIFIER and
        _peek_next().type == ASSIGN):
        return _assignment()
    # bare expression statement
    node = self._expr()
    _expect(SEMICOLON, "Expected ';'")
    return node
```

Unary minus desugaring

```
# -x --> BinaryOp('-', Literal(0), x)
# -.x --> BinaryOp('-', Literal(0.0), x)
# Done in _factor(). No UnaryOp node exists!
if tok.type == MINUS:
    op = advance()
    operand = _factor() # recursive
    return BinaryOp('-', Literal(0,...), operand, ...)
```

Error format: [line X, col Y]
ParseError: msg

AST Nodes – `ast_nodes.py`

```
# All inherit ASTNode (has line, column)
@dataclass class Program(ASTNode):
    statements: list[ASTNode]
@dataclass class Assign(ASTNode):
    name: str; value: ASTNode
@dataclass class If(ASTNode):
    condition: ASTNode
    then_block: Block
    else_block: Optional[Block]
@dataclass class While(ASTNode):
    condition: ASTNode; body: Block
@dataclass class FunctionDef(ASTNode):
    name: str
    params: list[str] # names only
    body: Block
@dataclass class Return(ASTNode):
    expr: ASTNode
@dataclass class Block(ASTNode):
    statements: list[ASTNode]
@dataclass class BinaryOp(ASTNode):
    op: str # '+', '-', '=', etc.
    left: ASTNode; right: ASTNode
@dataclass class Literal(ASTNode):
    value: int | float | bool | str
@dataclass class Identifier(ASTNode):
    name: str
@dataclass class FunctionCall(ASTNode):
    name: str; args: list[ASTNode]
```

ASTPrinter visitor pattern

```
# Dispatches via method name lookup:
def _visit(self, node, lines, level):
    m = "_visit_" + type(node).__name__
    getattr(self, m, self._visit_unknown)(
        node, lines, level)

# Add per new node class:
def _visit_NewNode(self, node, lines, level):
    ind = self._pad(level) # " " * level
    lines.append(f"{ind}NewNode")
    self._visit(node.child, lines, level+1)
```

Type Checker

`_FunctionContext` (dataclass!)

```
# Not a dict -- it's a dataclass:
@dataclass
class _FunctionContext:
    name: str
    return_type: LanguageType | None = None
    # Threaded through ALL _check_* calls:
    # _check_statement(node, scope, ctx)
    # _check_block(block, scope, ctx)
```

Type rules

Op	Requires	Out
+, -, *, /	INT + INT	INT
+, -, *, /, .	FLT + FLT	FLT
==, !=	same numeric	BOOL
if/while	BOOL cond	-
assign	value = var type	-

_infer_binary_type structure

```
def _infer_binary_type(self, node, lt, rt):
    :
    if node.op in {'+', '-', '*', '/'}:
        if lt != INTEGER or rt != INTEGER:
            ERROR
        return INTEGER
    if node.op in {'+', '-', '.', '*', '/'}:
        :
        if lt != FLOAT or rt != FLOAT:
            ERROR
        return FLOAT
    if node.op in {'==', '!='}:
        if not _is_numeric(lt): ERROR
        # lt
        if not _is_numeric(rt): ERROR
        # rt
        if lt != rt: ERROR
        return BOOLEAN
    raise TypeCheckError(...)
```

Return path analysis

```
@staticmethod
def _definitely_returns(block) -> bool:
    for stmt in block.statements:
        if isinstance(stmt, Return):
            return True
        if isinstance(stmt, If) and \
            stmt.else_block is not None:
```

```
if (_definitely_returns(
    stmt.then_block)
    and
    _definitely_returns(
    stmt.else_block)):
    return True
return False
# While loops do NOT guarantee return!
```

Error format: [line X, col Y]
TypeError: msg

Interpreter

Class structure

```
class Interpreter:
    _BUILTIN_PRINT = "print"
    # self._functions = {} # name -> runtime
    def __init__(self, output_callback=None):
    def execute(program)
    def _execute_statement(node, env)
    def _execute_block(block, env)
    def _evaluate(node, env) -> object
    def _evaluate_binary(n, lv, rv) -> object
    def _call_function(node, env) -> object
    def _call_builtin_print(node, env)
    @staticmethod _format_value(v) -> str
    @staticmethod _runtime_type(v) -> LT
```

```
class _Environment:
    # standalone
    def __init__(self, parent=None)
    def define(name, value) # current scope
    def get(name) -> object # searches chain
    def assign(name, value) # chain; create if not found
```

Return signal (exception pattern)

```
class _ReturnSignal(Exception):
    def __init__(self, val): self.value = val
    # _execute_statement, Return node:
    raise _ReturnSignal(
        self._evaluate(node.expr, env))
    # _call_function:
    try:
        self._execute_block(body, local_env)
    except _ReturnSignal as s:
        return s.value
```

Operators

```
'+' -> lv + rv
'- ' -> lv - rv
'* ' -> lv * rv
'/ ' -> int(lv)//int(rv) # FLOOR division
'+.' -> float(lv)+float(rv)
'-.' -> float(lv)-float(rv)
'*.' -> float(lv)*float(rv)
'/.' -> float(lv)/float(rv) # true div
'==' -> lv == rv # -> bool
'!=' -> lv != rv # -> bool
# Zero check before / and /.
```

print() built-in

```
# Output: "value : TypeName"
# print(42) -> "42 : Integer"
# print(3.14) -> "3.14 : Float"
# print(true) -> "true : Boolean"
# print("hi") -> "'hi' : String"
# Returns VOID; cannot assign result
```

Error format: [line X, col Y]
RuntimeError: msg

Language Examples

Basic

```
x = 10; y = 3;
z = x + y;
print(z); // 13: Integer
w = x / y;
print(w); // 3: Integer
```

```
a = 1.5;
b = 2.5;
c = a +. b;
print(c); // 4.0: Float
```

If-else

```
x = 5;
```

```
if (x != 0) {
    print("ok");
} else {
    print("zero");
}
```

While + func

```
def fact(n) {
```

```
r = 1; i = n;
while (i != 0) {
    r = r * i;
    i = i - 1;
}
return r;
print(fact(5)); // 120
```

Float

Trace & Predict Questions

Q: Token stream for x = 1 +. 2.0;

1. IDENTIFIER "x" 2. ASSIGN "=" 3. INT_LITERAL "1"
4. PLUS_DOT "+." 5. FLOAT_LITERAL "2.0" 6. SEMICOLON ";" 7. EOF

Lexer OK. Type Checker raises TypeError: +. requires FLOAT+FLOAT but left operand is INTEGER.

Q: AST for z = 3 + 4 * 2;

```
Assign 'z'
  BinaryOp '+' # _additive()
    Literal 3
    BinaryOp '*' # _term() runs first
      Literal 4
      Literal 2
  # Result: z = 3 + (4*2) = 11
```

Q: Token stream for x = 3.14; – why one token?

1. IDENTIFIER "x" 2. ASSIGN "=" 3. FLOAT_LITERAL "3.14" 4. SEMICOLON ";" 5. EOF

_number() sees 3, then . followed by 1 (digit), so continues scanning into one FLOAT_LITERAL. The _ARITH_DOT rule only fires when char-after-dot is NOT a digit.

Q: What does -x produce in the AST?

No UnaryOp node. Desugared in _factor():

```
# -x becomes:
BinaryOp(op='-', left=Literal(0),
         right=Identifier('x'))
# -.x becomes:
BinaryOp(op='-.', left=Literal(0.0),
         right=Identifier('x'))
```

Q: Type-check trace: `def f(a){return a+1;}; f(5);`

1. `_register_function(f)` – placeholder stored
2. `f(5)`: arg 5 is INTEGER $\rightarrow$ param a = INTEGER
3. Body: `a+1` – both INT, result INT
4. Return type = INTEGER; `_definitely_returns` = True
5. FunctionSymbol: return=INTEGER,
params=[(a,INTEGER)]

Q: `_definitely_returns` for: `def f(x){if(x!=0){return 1;}}`

1. Scan block statements: find If node
 2. If has `else_block`? No (None)
 3. No top-level Return found
 4. Returns **False** $\rightarrow$ TypeError: missing return
- Fix: add `else { return 0; }.`

Q: Interpreter environment chain for globals + function

```
g = 10;
def f(a) { return a + g; }
r = f(5);
```

1. Global env: {g=10, f=...}
2. `f(5)`: local_env = child of global env
3. local_env: {a=5}, parent=global_env
4. `a+g`: a=5 (local), g=10 (via parent chain)
5. Returns 15. `r=15` in global env.

Q: Predict output of `print(7/2);` and `print(-7/2);`

`7/2`: `int(7)//int(2) = 3  $\rightarrow$  3 : Integer`
`-7/2`: desugars to `BinaryOp('-', 0, 7)/2`.
 Wait – `-7` desugars to `0-7=-7`, then `-7/2`:
`int(-7)//int(2) = -4` (Python floor) $\rightarrow$ **-4 : Integer**
 (Truncation would give -3 – this is a known language quirk.)

Q: What errors occur and at what stage?

```
x = 5;
x = "hello";
if (x + 1) { print(x); }
```

Line 2: TypeError – Cannot assign String to Integer variable x. Type checker stops here (unless multi-error mode).
 Line 3 would also fail: condition `x+1` is INTEGER not BOOLEAN.

Q: What error for recursive call?

```
def fact(n) {
    return n * fact(n - 1);
}
```

TypeError: recursive function calls not supported. Detected in `_check_function_body` or `_infer_function_call_type` when `fact` calls `fact` within its own body check.

Implementation Scenarios – All File Changes

[+] **ADD** new code [~] **MODIFY** existing code [=] **WHERE** shows location in file (do not change) **Rule:** new keyword $\rightarrow$ `tokens.py` enum
 + `lexica.py` KEYWORDS dict

Q: FOR loop: `for (i=0; i!=10; i=i+1) {...}`

All 7 files: `tokens` $\rightarrow$ `lexica` $\rightarrow$ `ast_nodes` $\rightarrow$ `ast_printer`
 $\rightarrow$ `parser` $\rightarrow$ `type_checker` $\rightarrow$ `interpreter`

[+] ADD to: `tokens.py` - TokenType enum body

```
FOR = auto()
```

[+] ADD to: `lexica.py` - KEYWORDS dict

```
"for": TokenType.FOR,
```

[+] ADD to: `ast_nodes.py`

```
@dataclass
class For(ASTNode):
    init: Assign      # i = 0
    condition: ASTNode # i != 10
    update: Assign    # i = i + 1
    body: Block
```

[+] ADD to: `ast_printer.py` - new visitor method

```
def _visit_For(self, node, lines, level):
    ind = self._pad(level)
    lines.append(f"{ind}For")
    self._visit(node.init, lines, level+1)
    lines.append(f"{self._pad(level+1)}cond:")
    self._visit(node.condition, lines, level+2)
    lines.append(f"{self._pad(level+1)}update:")
    self._visit(node.update, lines, level+2)
    self._visit(node.body, lines, level+1)
```

[+] ADD to: `parse_tree_printer.py` - `_statement_node()` before While

```
if isinstance(node, For):
    return ParseTreeNode("statement",
                        [self._for_node(node)])
```

[+] ADD to: `parser.py` - `_statement()` before the WHILE check

```
if tok.token_type == TokenType.FOR:
    return self._for_stmt()
```

[*] ADD to: parser.py - new method _for_stmt()

```
def _for_stmt(self) -> For:
    tok = self._advance() # 'for'
    self._expect(LPAREN, "Expected '('")
    # init assignment
    id_t = self._expect(IDENTIFIER,
                        "Expected variable")
    self._expect(ASSIGN, "Expected '='")
    init_val = self._expr()
    self._expect(SEMICOLON, "Expected ';'")
    init = Assign(id_t.lexeme, init_val,
                 line=tok.line,
                 column=tok.column)

    # condition
    cond = self._expr()
    self._expect(SEMICOLON, "Expected ';'")
    # update assignment
    upd_t = self._expect(IDENTIFIER,
                        "Expected variable")
    self._expect(ASSIGN, "Expected '='")
    upd_val = self._expr()
    self._expect(RPAREN, "Expected ')'")
    update = Assign(upd_t.lexeme, upd_val,
                   line=tok.line,
                   column=tok.column)

    body = self._block()
    return For(init, cond, update, body,
              line=tok.line, column=tok.column)
```

[*] ADD to: type_checker.py - _check_statement() after While block

```
if isinstance(node, For):
    self._check_statement(
        node.init, scope, function_context)
    ct = self._infer_expr_type(
        node.condition, scope)
    if ct != LanguageType.BOOLEAN:
        raise TypeCheckError(
            self._error_at(node.condition,
                           "For condition must be Boolean"))
    self._check_statement(
        node.update, scope, function_context)
    self._check_block(
        node.body, scope.child_scope(),
        function_context)
    return
```

[*] ADD to: interpreter.py - _execute_statement() after While block

```
if isinstance(node, For):
    loop_env = _Environment(parent=environment)
    self._execute_statement(node.init, loop_env)
    while True:
        cond = self._evaluate(
            node.condition, loop_env)
        if not cond:
            break
        self._execute_block(
            node.body,
            _Environment(parent=loop_env))
        self._execute_statement(
            node.update, loop_env)
    return
```

Q: BREAK statement inside while/for

[*] ADD to: tokens.py

```
BREAK = auto()
```

[*] ADD to: lexica.py - KEYWORDS

```
"break": TokenType.BREAK,
```

[*] ADD to: ast_nodes.py

```
@dataclass
class Break (ASTNode):
    pass
```

[*] ADD to: ast_printer.py

```
def _visit_Break(self, node, lines, level):
    lines.append(self._pad(level) + "Break")
```

[*] ADD to: parser.py - _statement() before assignment check

```
if tok.token_type == TokenType.BREAK:
    t = self._advance() # consume 'break'
    self._expect(SEMICOLON, "Expected ';'")
    return Break(line=t.line, column=t.column)
```

Extend _FunctionContext to track loops:

[*] MODIFY in: type_checker.py - _FunctionContext dataclass

```
@dataclass
class _FunctionContext:
    name: str
    return_type: LanguageType | None = None
    in_loop: bool = False # <<< ADD
```

[*] ADD to: type_checker.py - _check_statement() after While block

```
if isinstance(node, Break):
    if (function_context is None or
        not function_context.in_loop):
        raise TypeCheckError(
            self._error_at(node,
                           "'break' outside of a loop"))
    return
```

[*] MODIFY in: type_checker.py - While handling: pass in_loop context

```
if isinstance(node, While):
    ct = self._infer_expr_type(
        node.condition, scope)
    if ct != LanguageType.BOOLEAN:
        raise TypeCheckError(...)
    # Build loop context:
    loop_ctx = _FunctionContext(
        name=(function_context.name
              if function_context else ""),
        return_type=(
            function_context.return_type
            if function_context else None),
        in_loop=True) # <<< KEY
    self._check_block(
        node.body,
        scope.child_scope(),
        loop_ctx) # <<< use loop_ctx
    return
```

[*] ADD to: interpreter.py - add exception class (top of file)

```
class _BreakSignal(Exception):
    pass
```

[*] ADD to: interpreter.py - _execute_statement() after While block

```
if isinstance(node, Break):
    raise _BreakSignal()
```

[*] MODIFY in: interpreter.py - While handler: wrap body in try/except

```
if isinstance(node, While):
    while True:
        cond = self._evaluate(
            node.condition, environment)
        if not isinstance(cond, bool): ...
        if not cond:
            break
        try:
            self._execute_block(
                node.body,
                _Environment(parent=environment))
        except _BreakSignal: # <<< ADD
            break # exit loop
    return
```

Q: CONTINUE statement (skip to next iteration)

Same pattern as break – only differences shown:

[+] ADD to: tokens.py + lexica.py KEYWORDS

```
# tokens.py:
CONTINUE = auto()
# lexica.py KEYWORDS:
"continue": TokenType.CONTINUE,
```

[+] ADD to: ast_nodes.py + ast_printer.py

```
# ast_nodes.py:
@dataclass
class Continue(ASTNode):
    pass

# ast_printer.py:
def _visit_Continue(self, node, lines, level):
    lines.append(self._pad(level) + "Continue")
```

[+] ADD to: parser.py - _statement()

```
if tok.token_type == TokenType.CONTINUE:
    t = self._advance()
    self._expect(SEMICOLON, "Expected ';'")
    return Continue(line=t.line,
                    column=t.column)
```

[+] ADD to: type_checker.py - _check_statement()

```
if isinstance(node, Continue):
    if (function_context is None or
        not function_context.in_loop):
        raise TypeCheckError(
            self._error_at(node,
                            "'continue' outside of a loop"))
    return
```

[+] ADD to: interpreter.py - exception + statement handler

```
class _ContinueSignal(Exception):
    pass

# In _execute_statement():
if isinstance(node, Continue):
    raise _ContinueSignal()
```

[~] MODIFY in: interpreter.py - While: also catch ContinueSignal

```
try:
    self._execute_block(
        node.body,
        _Environment(parent=environment))
except _BreakSignal:
    break
except _ContinueSignal:  # <<< ADD
    continue             # next iteration
```

Q: AND / OR operators (new precedence levels)

Key: insert _or() and _and() into parser call chain and update ALL entry points.

[+] ADD to: tokens.py

```
AND = auto()
OR  = auto()
```

[+] ADD to: lexica.py - KEYWORDS

```
"and": TokenType.AND,
"or":  TokenType.OR,
```

No new AST node – reuse BinaryOp('and',...) / BinaryOp('or',...).

[+] ADD to: parser.py - TWO new methods

```
def _or(self) -> ASTNode:
    """or: lowest new precedence"""
    node = self._and()
    while self._check(TokenType.OR):
        op = self._advance()
        right = self._and()
        node = BinaryOp(op.lexeme, node, right,
                        line=op.line,
                        column=op.column)

    return node

def _and(self) -> ASTNode:
    """and: above or, below == !="""
    node = self._expr() # comparisons
    while self._check(TokenType.AND):
        op = self._advance()
        right = self._expr()
        node = BinaryOp(op.lexeme, node, right,
                        line=op.line,
                        column=op.column)

    return node
```

[~] MODIFY in: parser.py - update ALL 6 call sites from _expr() to _or()

```
# _statement() bare expr: self._or()
# _assignment() value: self._or()
# _if_stmt() condition: self._or()
# _while_stmt() condition: self._or()
# _return_stmt() expr: self._or()
# _args() loop: self._or()
# (everywhere _expr() was called directly)
```

[+] ADD to: type_checker.py - _infer_binary_type() before final raise

```
if node.op in {'and', 'or'}:
    if lt != LanguageType.BOOLEAN:
        raise TypeCheckError(self._error_at(
            node, f"'{node.op}' needs Boolean left"))
    if rt != LanguageType.BOOLEAN:
        raise TypeCheckError(self._error_at(
            node, f"'{node.op}' needs Boolean right"))
    return LanguageType.BOOLEAN
```

[+] ADD to: interpreter.py - _evaluate_binary() before final raise

```
if node.op == 'and':
    return bool(left_value) and bool(right_value)
if node.op == 'or':
    return bool(left_value) or bool(right_value)
```

Q: NOT operator (unary boolean negation)

[+] ADD to: tokens.py + lexica.py KEYWORDS

```
# tokens.py: NOT = auto()
# lexica.py: "not": TokenType.NOT,
```

[+] ADD to: ast_nodes.py

```
@dataclass
class UnaryOp(ASTNode):
    op: str # 'not'
    operand: ASTNode
```

[+] ADD to: ast_printer.py

```
def _visit_UnaryOp(self, node, lines, level):
    ind = self._pad(level)
    lines.append(f"{ind}UnaryOp '{node.op}'")
    self._visit(node.operand, lines, level+1)
```

[+] ADD to: parser.py - _factor(): at TOP before minus checks

```
if tok.token_type == TokenType.NOT:
    op = self._advance()
    operand = self._factor()
    return UnaryOp('not', operand,
                  line=op.line,
                  column=op.column)
```

[+] ADD to: type_checker.py - _infer_expr_type(): new isinstance branch

```
if isinstance(node, UnaryOp):
    if node.op == 'not':
        t = self._infer_expr_type(
            node.operand, scope)
        if t != LanguageType.BOOLEAN:
            raise TypeCheckError(
                self._error_at(node,
                    "'not' requires Boolean operand"))
        return LanguageType.BOOLEAN
    raise TypeCheckError(self._error_at(
        node, f"Unknown unary op: {node.op}"))
```

[+] ADD to: interpreter.py - _evaluate(): new isinstance branch

```
if isinstance(node, UnaryOp):
    val = self._evaluate(node.operand,
                        environment)

    if node.op == 'not':
        return not bool(val)
    raise InterpreterError(...)
```

Q: Comparison operators <, >, <=, >=

[+] ADD to: tokens.py

```
LESS = auto()
LESS_EQUAL = auto()
GREATER = auto()
GREATER_EQUAL = auto()
```

[+] ADD to: lexica.py - _scan_token() after the ! case

```
if char == '<':
    if self._match('='):
        return self._make_token(
            TokenType.LESS_EQUAL)
    return self._make_token(TokenType.LESS)
if char == '>':
    if self._match('='):
        return self._make_token(
            TokenType.GREATER_EQUAL)
    return self._make_token(TokenType.GREATER)
```

[~] MODIFY in: parser.py - _expr(): extend while condition

```
while (self._check(TokenType.EQUAL_EQUAL) or
self._check(TokenType.BANG_EQUAL) or
self._check(TokenType.LESS) or # <<<
self._check(TokenType.LESS_EQUAL) or # <<<
self._check(TokenType.GREATER) or # <<<
self._check(TokenType.GREATER_EQUAL)): # <<<
    op_tok = self._advance()
    right = self._additive()
    node = BinaryOp(op_tok.lexeme, node,
                  right, line=op_tok.line,
                  column=op_tok.column)
```

[+] ADD to: type_checker.py - _infer_binary_type() after == block

```
if node.op in {'<', '>', '<=', '>='}:
    if lt not in {LanguageType.INTEGER,
        LanguageType.FLOAT}:
        raise TypeCheckError(
            self._error_at(node,
                f"'{node.op}' requires numeric"))
    if lt != rt:
        raise TypeCheckError(
            self._error_at(node,
                f"'{node.op}' requires same type"))
    return LanguageType.BOOLEAN
```

[+] ADD to: interpreter.py - _evaluate_binary()

```
if node.op == '<': return left_value < right_value
if node.op == '>': return left_value > right_value
if node.op == '<=': return left_value <= right_value
if node.op == '>=': return left_value >= right_value
```

Q: MODULO % for integers

[+] ADD to: tokens.py

```
PERCENT = auto()
```

[+] ADD to: lexica.py - _scan_token() after existing single-char ops

```
if char == '%':
    return self._make_token(TokenType.PERCENT)
```

[~] MODIFY in: parser.py - _term(): add PERCENT to while

```
while (self._check(TokenType.STAR) or
self._check(TokenType.SLASH) or
self._check(TokenType.STAR_DOT) or
self._check(TokenType.SLASH_DOT) or
self._check(TokenType.PERCENT)): # <<<
    op_tok = self._advance()
    right = self._factor()
    node = BinaryOp(op_tok.lexeme, node,
                  right, ...)
```

[+] ADD to: type_checker.py - _infer_binary_type()

```
if node.op == '%':
    if lt != LanguageType.INTEGER or \
        rt != LanguageType.INTEGER:
        raise TypeCheckError(self._error_at(
            node, "'%' requires Integer operands"))
    return LanguageType.INTEGER
```

[+] ADD to: interpreter.py - _evaluate_binary()

```
if node.op == '%':
    if int(right_value) == 0:
        raise InterpreterError(
            self._error_at(node, "Modulo by zero"))
    return int(left_value) % int(right_value)
```

Q: POWER ** operator (higher than *)

Key: insert `_power()` between `_term()` and `_factor()`.

[*] ADD to: tokens.py

```
STAR_STAR = auto()    # **
```

[*] ADD to: lexica.py - `_scan_token()`: intercept ** BEFORE `_ARITH_DOT`

```
# Add this check BEFORE the _ARITH_DOT lookup:
if char == '*' and self._match('*'):
    return self._make_token(
        TokenType.STAR_STAR)
# then fall through to _ARITH_DOT for single *
```

[*] ADD to: parser.py - new method (between `_term` and `_factor`)

```
def _power(self) -> ASTNode:
    node = self._factor()
    while self._check(TokenType.STAR_STAR):
        op = self._advance()
        right = self._factor()    # left-assoc
        node = BinaryOp('*', node, right,
                        line=op.line,
                        column=op.column)

    return node
```

[*] MODIFY in: parser.py - `_term()`: change `_factor()` calls to `_power()`

```
def _term(self) -> ASTNode:
    node = self._power()        # was _factor()
    while ...:
        ...
        right = self._power()  # was _factor()
```

[*] ADD to: type_checker.py + interpreter.py

```
# type_checker.py -- _infer_binary_type():
if node.op == '**':
    if lt != LanguageType.INTEGER or \
       rt != LanguageType.INTEGER:
        raise TypeCheckError(...)
    return LanguageType.INTEGER

# interpreter.py -- _evaluate_binary():
if node.op == '**':
    return int(left_value) ** int(right_value)
```

Q: ELIF chain (desugar – no new AST node)

[*] ADD to: tokens.py + lexica.py

```
# tokens.py: ELIF = auto()
# lexica.py: "elif": TokenType.ELIF,
```

[*] MODIFY in: parser.py - `_if_stmt()`: add elif branch

```
def _if_stmt(self) -> If:
    tok = self._advance()    # 'if'
    self._expect(LPAREN, "Expected '('")
    condition = self._expr()
    self._expect(RPAREN, "Expected ')'")
    then_block = self._block()
    else_block = None
    if self._check(TokenType.ELIF):    # <<< ADD
        # Desugar: elif -> else { if ... }
        elif_node = self._if_stmt()    # recursive
        else_block = Block([elif_node],
                          line=self._peek().line,
                          column=0)
    elif self._check(TokenType.ELSE):
        self._advance()
        else_block = self._block()
    return If(condition, then_block,
             else_block, line=tok.line,
             column=tok.column)
```

Nothing else changes. TypeChecker and Interpreter already handle nested If.

Q: Single-line comments // (lexer only)

[*] MODIFY in: lexica.py - `_scan_token()`: before `_ARITH_DOT` lookup

```
# Intercept '/' BEFORE the _ARITH_DOT dict:
if char == '/':
    if self._match('/'):    # it's a comment
        while (not self._is_at_end() and
               self._peek() != '\n'):
            self._advance()    # skip to end of line
        return None           # emit no token
    # else: fall through -- '/' handled by
    # _ARITH_DOT for SLASH / SLASH_DOT
```

No other files change.

Q: String concatenation ^^ operator

[*] ADD to: tokens.py

```
CARET_CARET = auto()
```

[*] ADD to: lexica.py - `_scan_token()`

```
if char == '^':
    if self._match('^'):
        return self._make_token(
            TokenType.CARET_CARET)
    raise LexerError(self._error_message(
        "Unexpected '^'. Did you mean '^'?"))
```

[*] MODIFY in: parser.py - `_additive()`: add to while condition

```
while (self._check(TokenType.PLUS) or
      ...
      self._check(TokenType.CARET_CARET)):    # <<<
    op_tok = self._advance()
    right = self._term()
    node = BinaryOp(op_tok.lexeme, node, right, ...)
```

[*] ADD to: type_checker.py - `_infer_binary_type()`

```
if node.op == '^':
    if lt != LanguageType.STRING or \
       rt != LanguageType.STRING:
        raise TypeCheckError(self._error_at(
            node, "'^' requires String operands"))
    return LanguageType.STRING
```

[*] ADD to: interpreter.py - `_evaluate_binary()`

```
if node.op == '^':
    return str(left_value) + str(right_value)
```

Q: Extend ==/!= to Strings and Booleans

Only type_checker.py changes:

[*] MODIFY in: type_checker.py - == / != block in `_infer_binary_type()`

```
if node.op in {'==', '!='}:
    # Original: only numeric types allowed
    # if not self._is_numeric(lt): ERROR
    # New: allow String and Boolean too:
    comparable = {LanguageType.INTEGER,
                  LanguageType.FLOAT,
                  LanguageType.STRING,    # <<<
                  LanguageType.BOOLEAN}  # <<<
    if lt not in comparable:
        raise TypeCheckError(self._error_at(
            node, f"Cannot compare {lt.value}"))
    if lt != rt:
        raise TypeCheckError(self._error_at(node,
            f"'{node.op}' requires same type, "
            f"got {lt.value} and {rt.value}"))
    return LanguageType.BOOLEAN
```

Interpreter: no change – Python ==/!= already works on strings and bools.

Q: Built-in functions: int_of, float_of, len, abs, input

Only type_checker.py and interpreter.py:

```
[*] ADD to: type_checker.py - _infer_function_call_type()

if node.name == 'int_of':
    if len(node.args) != 1: raise ...
    t = self._infer_expr_type(node.args[0], scope)
    if t != LanguageType.FLOAT:
        raise TypeCheckError(self._error_at(
            node, "int_of() needs Float argument"))
    return LanguageType.INTEGER

if node.name == 'float_of':
    if len(node.args) != 1: raise ...
    t = self._infer_expr_type(node.args[0], scope)
    if t != LanguageType.INTEGER:
        raise TypeCheckError(self._error_at(
            node, "float_of() needs Integer arg"))
    return LanguageType.FLOAT

if node.name == 'len':
    if len(node.args) != 1: raise ...
    t = self._infer_expr_type(node.args[0], scope)
    if t != LanguageType.STRING:
        raise TypeCheckError(self._error_at(
            node, "len() needs String argument"))
    return LanguageType.INTEGER

if node.name == 'abs':
    if len(node.args) != 1: raise ...
    t = self._infer_expr_type(node.args[0], scope)
    if t != LanguageType.INTEGER:
        raise TypeCheckError(self._error_at(
            node, "abs() needs Integer argument"))
    return LanguageType.INTEGER

if node.name == 'input':
    if len(node.args) != 0:
        raise TypeCheckError(self._error_at(
            node, "input() takes no arguments"))
    return LanguageType.STRING
```

[*] ADD to: interpreter.py - _call_function() before _functions lookup

```
if node.name == 'int_of':
    return int(self._evaluate(
        node.args[0], environment))
if node.name == 'float_of':
    return float(self._evaluate(
        node.args[0], environment))
if node.name == 'len':
    return len(str(self._evaluate(
        node.args[0], environment)))
if node.name == 'abs':
    return abs(int(self._evaluate(
        node.args[0], environment)))
if node.name == 'input':
    return input()
```

Q: Compound assignment += and -= (desugar)

Strategy: parse $x \ += \ e$ into $\text{Assign}(x, \text{BinaryOp}(+, x, e))$ – no new AST node.

[*] ADD to: tokens.py

```
PLUS_EQUAL = auto() # +=
MINUS_EQUAL = auto() # -=
```

[*] ADD to: lexica.py - _scan_token(): BEFORE _ARITH_DOT lookup

```
# Check += and -= before falling to _ARITH_DOT:
if char == '+' and self._match('='):
    return self._make_token(TokenType.PLUS_EQUAL)
if char == '-' and self._match('='):
    return self._make_token(TokenType.MINUS_EQUAL)
# Then _ARITH_DOT handles +/- normally
```

[*] ADD to: parser.py - _statement(): add compound check

```
# Check compound assignment: IDENT += expr ;
_compound = {TokenType.PLUS_EQUAL: '+',
              TokenType.MINUS_EQUAL: '-'}
if (tok.token_type == TokenType.IDENTIFIER and
    self._peek_next().token_type
    in _compound):
    name_tok = self._advance()
    op_tok = self._advance() # += or -=
    base_op = _compound[op_tok.token_type]
    rhs = self._expr()
    self._expect(SEMICOLON, "Expected ';'")
    # Desugar: x += e -> Assign(x, x + e)
    lhs_id = Identifier(name_tok.lexeme,
                        line=name_tok.line,
                        column=name_tok.column)
    binary = BinaryOp(base_op, lhs_id, rhs,
                    line=op_tok.line,
                    column=op_tok.column)
    return Assign(name_tok.lexeme, binary,
                line=name_tok.line,
                column=name_tok.column)
```

No type_checker or interpreter changes – desugars to existing Assign(BinaryOp).

Q: Modify print() output (remove type tag)

[*] MODIFY in: interpreter.py - _call_builtin_print()

```
def _call_builtin_print(self, node, env):
    if len(node.args) != 1:
        raise InterpreterError(...)
    arg_val = self._evaluate(node.args[0], env)
    formatted = self._format_value(arg_val)
    # ORIGINAL (with type tag):
    # type_name = self._runtime_type(
    #     arg_val).value
    # output = f"{formatted} : {type_name}"
    # NEW (value only):
    output = formatted # <<< CHANGE
    self._output(output)
```

No other files change.

Q: Fix truncation: -7/2 should be -3 not -4

[*] MODIFY in: interpreter.py - _evaluate_binary() '/' case

```
if node.op == '/':
    if int(right_value) == 0:
        raise InterpreterError(
            self._error_at(node, "Division by zero"))
    # ORIGINAL (Python floor division):
    # return int(left_value) // int(right_value)
    # FIXED (truncation towards zero):
    return int(int(left_value) / # <<< CHANGE
              int(right_value))
```

Why: $\text{int}(-7/2) = \text{int}(-3.5) = -3$; $-7//2 = -4$ (Python floor).

Change	Files (in order)	Error source	Stage
New keyword	tokens.py + lexica.py KEYWORDS	Bad char, unterminated str	Lexer
New symbol	tokens.py + lexica.py _scan_token	Missing ;) }	Parser
New statement	tokens → lexica → ast_nodes → ast_printer → parse_tree_printer → parser → type_checker → interpreter	Type mismatch, undef var	TypeChkr
		Division by zero	Interp.
New operator	tokens → lexica → parser → type_checker → interpreter	Op precedence	Method
New built-in	type_checker + interpreter only	or (if added)	_or()
Desugar syntax	parser only (no new node)	and (if added)	_and()
Change output	interpreter only	==, !=, <, >, <=, >=	_expr()
Change type rule	type_checker only	+, -, +., -., ^^	_additive()
		*, /, *, ., /., %	_term()
		** (if added)	_power()
		Unary -, -, ., not	_factor()

Critical Rules – Memorise These

- **KEYWORDS dict is in lexica.py**, not tokens.py
- **_FunctionContext is a dataclass** (not dict): fields name, return_type; add in_loop: bool = False for break/continue
- **ASTPrinter uses method-name dispatch: _visit_<ClassName>** – must add for every new AST node class
- **parse_tree_printer.py** uses isinstance chain in _statement_node() – add a new branch for each new statement node
- **Desugar in parser, not later:** elif→nested If, unary minus→BinaryOp, +=→Assign(BinaryOp) – type checker & interpreter get existing node types
- **New lower-precedence op:** add wrapper **above** _expr() and update ALL 6 call sites (_statement, _assignment, _if_stmt, _while_stmt, _return_stmt, _args)
- **New higher-precedence op:** insert new method **between** existing levels; change the level below to call the new one
- **Exception signals:** add class in interpreter, raise in _execute_statement, catch in loop handler (_BreakSignal, _ContinueSignal, _ReturnSignal)
- **All errors:** use self._error_at(node, msg) – produces [line X, col Y] ErrorType: msg
- **Function context:** must be passed through ALL recursive calls to _check_block and _check_statement for break/continue/return validation to work
- **Built-in functions:** add to _infer_function_call_type() in type_checker AND to _call_function() in interpreter – BEFORE the self._functions.get() lookup
- **Integer division is floor (//)**, not truncation – -7//2 = -4 not -3